

What is a Session?

Session represents conversational state between client and server and it can consist of multiple request and response between client and server. Since HTTP and Web Server both are stateless, the only way to maintain a session is when some unique information about the session is passed between server and client in every request and response.

HTTP protocol and Web Servers are stateless, what it means is that for web server every request is a new request to process and they can't identify if it's coming from client that has been sending request previously.

But sometimes in web applications, we should know who the client is and process the request accordingly. For example, a shopping cart application should know who is sending the request to add an item and in which cart the item has to be added or who is sending checkout request so that it can charge the amount to correct client.

What is the need of session tracking?

1. To identify the client among multiple clients
2. To remember the conversational state of the client (eg : list of the purchased books/ shopping cart/bank acct details/stocks) throughout current session

session = Represents duration or time interval
default session timeout for Tomcat = 30 minutes

Session consists of all requests/responses coming from/ sent to SAME client from login to logout or till session expiration timeout.

There are several techniques for session tracking.

Java EE specific techniques :

1. Plain Cookie based scenario
- OR
2. HttpSession interface

3. HttpSession + URL rewriting

Techniques

1. Plain Cookie based scenario

What is a cookie?

Cookie is small amount of text data.

Created by -- server (servlet or JSP prog or WC) & downloaded (sent) to client browser---within response header

Cookie represents data shared across multiple dynamic pages from the SAME web application. (meant for the same client)

Steps :

1. Create cookie/s instance/s
`javax.servlet.http.Cookie(String cName,String cVal)`

2. Add the cookie/s to the **resp** hdr.

HttpServletResponse API :

void addCookie(Cookie c)

3. To retrieve the cookies :

HttpServletRequest :

Cookie[] getCookies()

4. Cookie class methods :

String getName()

String getValue()

void setMaxAge(int ageInSeconds)

def age == -1 ---> browser stores cookie in cache

= 0 ---> clnt browser should delete cookie

> 0 --- persistent cookie --to be stored on clnt's hard disk.

int getMaxAge()

Disadvantages of pure cookie based scenario

0. Web developer (servlet prog) has to manage cookies.

1. Cookies can handle only text data : storing Java obj or bin data difficult.

2. As no of cookies inc., it will result into increased net traffic.

3. In cookie based approach : entire state of the clnt is saved on the clnt side. If the clnt browser rejects the cookies: state will be lost : session tracking fails.

How to redirect client automatically to next page ? (in the NEXT request)

API of HttpServletResponse

public void sendRedirect(String redirectLoc)

eg : resp.sendRedirect("s2");

IMPORTANT :

WC -- throws

java.lang.IllegalStateException: Cannot call sendRedirect() after the response has been committed(eg : pw.flush(),pw.close(...))

Technique # 2 : State management based on HttpSession API

In this technique :

Entire state of the client is not saved on client side , instead saved on the server side data structure (Http Sesion object) BUT the key to this Http Session object is STILL sent to client in form of a cookie.(cookie management is done by WC)

Servlet programmer can store/restore java objects directly under the session scope(API :

setAttribute/getAttribute)

Above mentioned , disadvantages ---0, 1 & 2 are reomved.

BUT entire session tracking again fails , if cookies are disabled.

Steps for javax.servlet.http.HttpSession i/f based state management

1. Get HttpSession object from WC

API of HttpServletRequest ---

```
public HttpSession getSession()
```

Meaning --- Servlet requests WC to either create n return a NEW HttpSession object(for new clnt) or ret the existing one from WC's heap for existing client.

HttpSession --- i/f from javax.servlet.http

In case of new client :

```
HttpSession<String,Object> --empty map
```

```
String,Object ---- (entry)= attribute
```

OR

```
HttpSession getSession(boolean create)
```

2. : How to save data in HttpSession?(scope=entire session)

API of HttpSession i/f

```
public void setAttribute(String attrName,Object attrVal)
```

eg : session.setAttribute("clnt_info",cust);//no javac err

attribute : server side object ---server side entry (key n value pair) --map

cust : Customer POJO ref.

equivalent to map.put(k,v)

eg : hs.setAttribute("cart",l1);

3. For getting session data(getting attributes)

```
public Object getAttribute(String attrName) //key
```

eg : Customer cust=(Customer) hs.getAttribute("clnt_info");

It returns Object so need to downcast

4. To get session ID , generated by WC , unique per client

```
public String getId()
```

4.5 How to remove attribute from the session scope?

```
public void removeAttribute(String attrName)
```

eg : hs.removeAttribute("clnt_info");

5. How to invalidate session?

HttpSession API

```
public void invalidate()
```

(WC marks HS object on the server side for GC ---BUT cookie is NOT deleted from clnt browser)

6. HttpSession API

```
public boolean isNew()
```

Retns true for new client & false for existing client.

7.How to find all attr names from the session ?

```
public Enumeration<String> getAttributeNames()
```

--rets java.util.Enumeration of attr names.

Enumeration is equivalent to Iterator<T>

8. Default session timeout value for Tomcat = 30 mins

How to change session timeout ?

HttpSession i/f method

public void setMaxInactiveInterval(int secs)

eg : hs.setMaxInactiveInterval(300); --for 5 mins .

OR via xml tags in web.xml

<session-config>

<session-timeout>minutes</session-timeout> : unit : min

</session-config>

NOTE :

What is an attribute ?

attribute = server side object(entry/mapping=key value pair)

who creates server side attrs ? -- web developer (servlet or JSP prog)

Each attribute has --- attr name(String) & attr value (java.lang.Object)

Attributes can exist in one of 3 scopes --- req. scope, session scope or application scope

1. Meaning of req scoped attr = attribute is visible for current req.

2. Meaning of session scoped attr = attribute is visible for current session.(shared across multiple reqs coming from SAME clnt)

3. Meaning of application scoped attr = attribute is visible for current web appln.(shared across multiple reqs from ANY clnt BUT for the SAME web application)